STLMC User Manual

Geunyeol Yu
Pohang University of Science and Technology, Pohang, Korea

October 2026
Version 1.0.2.dev1

Contents

1 Introduction **2**

2 Installation **2**
2.1 Native solver requirements . . . 3
2.2 Removing optional components . . . 3

3 Running example **3**

4 Modeling language **4**
4.1 Variable declarations . . . 5
4.2 Mode definitions . . . 6
4.3 Initial conditions . . . 7
4.4 State propositions and STL properties . . . 7
4.5 Comments . . . 8

5 Analysis **8**
5.1 Running an analysis . . . 8

6 Configuration **10**
6.1 Command line arguments . . . 10
6.2 Configuration files . . . 10

7 Counterexample and witness analysis **13**
7.1 Text output . . . 13
7.2 Visualization . . . 14

8 Browser workspace **16**
8.1 Starting the workspace . . . 17
8.2 Editing and running analyses . . . 17
8.3 Inspecting and keeping results . . . 17

9 SMT solver support **17**
9.1 Solver selection and compatibility . . . 18
9.2 dReal function names . . . 18

1 Introduction

STLmc is a bounded model checker based on SMT for signal temporal logic (STL) properties of hybrid systems [7]. It verifies whether all behaviors of a hybrid system satisfy an STL property up to a discrete bound, a time bound, and a robustness threshold specified by the user. The tool supports hybrid systems with discrete transitions and continuous dynamics, including nonlinear ordinary differential equations (ODEs).

STLmc provides a modeling language for hybrid automata and STL properties, a command line interface and a browser workspace for model checking and reachability analysis, and support for inspecting counterexamples and reachability witnesses. It can use Z3 [2], Yices2 [3], CVC5, or dReal [4] as its underlying SMT solver.

Figure 1 gives an overview of the STLmc architecture. The tool takes a hybrid automaton H, an STL property φ or a reachability target, a discrete bound N, a time bound τ, and a robustness threshold ϵ. The parser reads the model, property, and analysis configuration. The model checking engine reduces robust STL model checking to Boolean STL model checking through ϵ-strengthening and constructs bounded SMT queries. The queries can be solved directly or using an optionally parallelized two step procedure, which is particularly useful for ODE models.

Model checking results are reported as satisfied, violated, or unknown. A satisfied result means that the bounded counterexample search found no violation of the requested robustness requirement, subject to the solver's semantics. An unknown result is inconclusive. Reachability results are reported as reachable, unreachable within the given bounds, or unknown. The tool can export a violating execution or reachability witness as text or visualize it as signal and robustness plots.

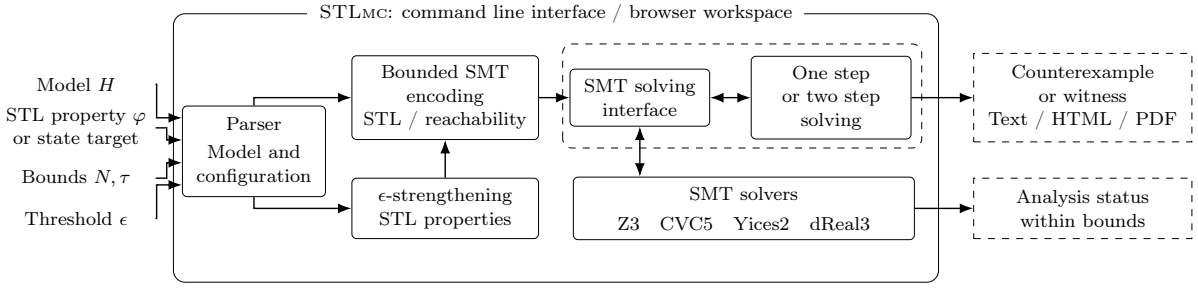


Figure 1: Architecture of STLmc.

The tool is available at `https://stlmc.github.io/`. Further details about the design, model checking algorithm, and evaluation are presented in the CAV 2022 tool paper [7].

2 Installation

STLmc supports CPython 3.11 through 3.14. Install it in a virtual environment with a current version of `pip`:

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ python -m pip install --upgrade pip
$ python -m pip install stlmc
```

For the development version described here, use `python -m pip install --pre stlmc`. Building a wheel from source additionally requires Ruby, Bundler, and the locked Jekyll gems for the GUI. These tools are not required when installing a wheel.

The package provides four commands: `stlmc` runs analyses, `stlmc-vis` inspects results, `stlmc-installer` manages optional components, and `stlmc-gui` opens a browser workspace.

Solver bindings and plotting packages are installed separately. Install and check Z3 for a first analysis:

```
$ stlmc-installer install --solver z3
$ stlmc-installer check --solver z3
$ stlmc --version
```

Other SMT solvers are selected with `cvc5`, `yices`, or `dreal`. Internal reasoning uses Z3 by default independently of the SMT solver, so keep Z3 installed when using these defaults. Multiple names may be supplied together:

```
$ stlmc-installer install --solver z3 cvc5 yices dreal
```

HTML plots require Bokeh, and PDF plots require Matplotlib. Install and check the desired renderers with:

```
$ stlmc-installer install --visualize html pdf
$ stlmc-installer check --visualize html pdf text
```

Text output uses the base package dependencies. GUI setup is described in Section 8. Running the installer without an action displays help. `check` inspects availability without installing components; `install --all` selects every optional component.

2.1 Native solver requirements

The installer downloads Yices2 2.7.0 into a user directory on supported Linux and macOS systems. Linux x86-64 with glibc 2.29 or newer also receives a managed CUDD dependency. The macOS archives require a compatible CUDD library; the ARM64 archive requires macOS 15 or newer and the Intel archive macOS 12 or newer. An explicit source build is available with `install --solver yices --build`. Parallel Yices2 analysis requires a native library verified as thread safe; nonlinear arithmetic requires MCSAT.

The default dReal3 download is the official 3.16.06.02 executable for Linux x86-64 or Intel macOS. Apple Silicon requires an existing Rosetta installation. This release has known ODE invariant limitations; a successful installation check does not certify general ODE correctness. A separate executable can be selected with `-executable-path`; see Section 6.

2.2 Removing optional components

Preview removal before uninstalling selected components:

```
$ stlmc-installer uninstall --solver yices --visualize pdf --dry-run
$ stlmc-installer uninstall --solver yices --visualize pdf
```

Removal preserves system installations, modified files, models, and results. Stop analyses using the selected components before removing them. Detailed platform requirements and troubleshooting are maintained in the installation guide.

3 Running example

This section introduces a networked thermostat model adapted from Bae and Gao [1], building on the classical thermostat hybrid automaton [5]. Two rooms are connected by an open door, and each has a heater that can be either on or off. For room $i \in \{0, 1\}$, let x_i denote its temperature and $q_i \in \{\mathsf{On}, \mathsf{Off}\}$ its heater mode. Heat transfer through the doorway couples the two rooms. The continuous dynamics of room i are given by:

$$\dot{x}_i = \begin{cases} k_i(h_i - (c_i x_i - d_i x_{1-i})) & (\mathsf{On}) \\ -k_i(c_i x_i - d_i x_{1-i}) & (\mathsf{Off}) \end{cases}$$

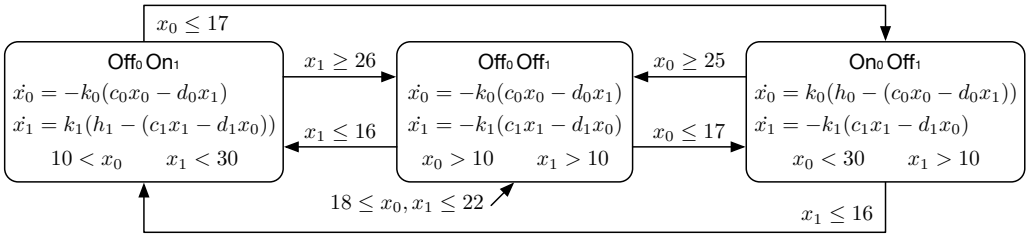


Figure 2: Hybrid automaton for the networked thermostat model.

The positive constants k_i, h_i, c_i, and d_i depend on the size of the room, the power of the heater, and the size of the open door.

Initially, both heaters are off and $18 \leq x_0, x_1 \leq 22$. Temperature guards enable transitions that turn heaters on or off; an enabled transition need not occur immediately. The three controller modes shown in Figure 2 ensure that at most one heater is on.

We use the following STL properties throughout the manual:

- $\mathsf{f1} = \Diamond_{[0,15]}((x_0 \geq 14)\,\mathbf{U}_{[0,\infty)}(x_1 \leq 19))$: there is a starting point within the first 15 time units from which the temperature of the first room stays at or above 14 until the second room reaches 19 or below. The latter condition must eventually hold within the analysis window and may already hold at the starting point.

- $\mathsf{f2} = \Box_{[2,4]}((x_0 - x_1 \geq 4) \rightarrow \Diamond_{[3,10]}(x_0 - x_1 \leq -3))$: between time 2 and time 4, whenever the temperature of the first room exceeds that of the second by at least 4, the temperature of the second exceeds that of the first by at least 3 after a delay of 3 to 10 time units.

- $\mathsf{f3} = \Box_{[0,10]}((x_0 > 23)\,\mathbf{R}_{[0,\infty)}(x_0 - x_1 \geq 4))$: at every point during the first 10 time units, the temperature of the first room exceeds that of the second by at least 4 through the point at which it exceeds 23. If it never exceeds 23, the difference remains at least 4 over the remaining analysis window. Unbounded temporal intervals are evaluated within the finite time window configured for an analysis.

The complete STLmc input for this example is presented in Section 4. Its analysis and configuration are described in Sections 5 and 6.

4 Modeling language

STLmc provides a modeling language for hybrid systems. Its model file format, inspired by dReach [6], consists of the following sections, which must appear in this order:

- *Variable declarations* define state and mode variables and constants.
- *Mode definitions* specify invariants, continuous flows, and jumps.
- *Initial conditions* define the initial states.
- *State propositions* optionally name conditions used in STL formulas.
- *STL properties* define temporal requirements over model states.

Figure 3 presents the complete STLmc model file for the running example. The file declares variables that record whether each heater is on or off, the two room temperatures, and the constants for their continuous dynamics. It also defines the invariants and switching behavior of three modes, the initial states, and the state propositions and STL properties.

```
int on0;              int on1;
[10, 35] x0;          [10, 35] x1;
const k0 = 0.015;     const k1 = 0.045;
const h0 = 100;       const h1 = 200;
const c0 = 0.98;      const c1 = 0.97;
const d0 = 0.01;      const d1 = 0.03;

{ mode: on0 = 0;      on1 = 0;
  inv:  x0 > 10;      x1 > 10;
  flow:
    d/dt[x0] = -k0 * (c0*x0 - d0*x1);
    d/dt[x1] = -k1 * (c1*x1 - d1*x0);
  jump:
    x0 <= 17 => (and (on0' = 1)
      (on1' = on1) (x0' = x0) (x1' = x1));
    x1 <= 16 => (and (on1' = 1)
      (on0' = on0) (x0' = x0) (x1' = x1));
}
{ mode: on0 = 0;      on1 = 1;
  inv:  10 < x0;      x1 < 30;
  flow:
    d/dt[x0] = -k0 * (c0*x0 - d0*x1);
    d/dt[x1] = k1 *
      (h1 - (c1*x1 - d1*x0));
  jump:
    x0 <= 17 => (and (on0' = 1)
      (on1' = 0) (x0' = x0) (x1' = x1));
    x1 >= 26 => (and (on1' = 0)
      (on0' = on0) (x0' = x0) (x1' = x1));
}
{ mode: on0 = 1;      on1 = 0;
  inv:  x0 < 30;      x1 > 10;
  flow:
    d/dt[x0] = k0 *
      (h0 - (c0*x0 - d0*x1));
    d/dt[x1] = -k1 * (c1*x1 - d1*x0);
  jump:
    x1 <= 16 => (and (on0' = 0)
      (on1' = 1) (x0' = x0) (x1' = x1));
    x0 >= 25 => (and (on0' = 0)
      (on1' = on1) (x0' = x0) (x1' = x1));
}
init: on0 = 0; 18 <= x0; x0 <= 22;
      on1 = 0; 18 <= x1; x1 <= 22;

proposition:
  [p1]: x0 - x1 >= 4;
  [p2]: x0 - x1 <= -3;


goal:
  [f1]: <>[0,15] (x0 >= 14 U[0,inf) x1 <= 19);
  [f2]: []<>[2,4] (p1 -> <>[3,10] p2);
  [f3]: []<>[0,10] (x0 > 23 R[0,inf) p1);
```

Figure 3: The model file `thm.model`.

4.1 Variable declarations

STLmc represents a hybrid state using mode and continuous variables. *Mode variables* remain constant during continuous evolution and may be updated by jumps, whereas *continuous variables* have real values and evolve according to the flow of the current mode. Together, their values describe a state of the hybrid automaton. A model may also declare named constants whose values do not change.

Mode variables may be declared with one of three types: `bool` variables with `true` and `false` values, `int` variables with integer values, and `real` variables with real values. For example, the following declares one mode variable of each type:

```
bool b;    int i;    real r;
```

Variable names and goal or proposition labels must begin with a letter and may contain only letters and digits. Language keywords cannot be used as identifiers; these include `t` and `inf`, which denote time and infinity, respectively.

Continuous variables are declared with domain intervals. Domains can be bounded or unbounded intervals of real numbers, including open, closed, and half open intervals. An infinite endpoint is written as `-inf` or `inf` with an open parenthesis. A singleton domain is written with one value in square brackets. For example, the following declares four continuous variables with different domains:

```
[0, 50] x;    (-1.1, 1) y;    (-inf, inf) z;    [3] w;
```

Constants are introduced with the `const` keyword. Constants can have the Boolean literals `true` and `false` or a numeric literal. Numeric literals may use integer, decimal, or scientific notation; an arithmetic expression such as `1/3` is not accepted in a constant declaration. For example, the following declares a numeric constant `k1` and a Boolean constant `heaterEnabled`:

```
const k1 = 0.015;    const heaterEnabled = true;
```

Mode variable, continuous variable, and constant declarations may be interleaved, but they must all precede the first mode block.

4.2 Mode definitions

In STLMC, a *mode block* describes one or more discrete configurations together with their invariant, continuous dynamics, and outgoing transitions. A model may contain multiple mode blocks, each with four components:

```
{
    mode: ...
    inv:  ...
    flow: ...
    jump: ...
}
```

The components must appear in the order shown. The `mode` and `flow` components contain at least one condition or equation. The `inv` and `jump` components may be empty.

A mode block applies to the current state valuations selected by its `mode` component. Its `inv` component gives conditions that must hold throughout continuous evolution, while its `flow` component defines that evolution. The `jump` component contains outgoing guard and reset rules.

A `mode` component contains Boolean state conditions separated by semicolons, normally over mode variables. The conjunction of these conditions represents a set of modes. Suppose that `b` is a Boolean mode variable and `i` is an integer mode variable. The following conditions select the two valuations $\{(true, 1), (true, 2)\}$:

```
b = true;    i > 0;    i < 3;
```

An `inv` component contains Boolean state conditions separated by semicolons, typically over continuous variables. The conjunction of these conditions is the invariant of the mode block and must hold throughout its continuous evolution. For example, the following declares an invariant condition for two continuous variables `x` and `y`:

```
x < 30;    y > -0.5;
```

A `flow` component contains either a system of ordinary differential equations (ODEs) or closed form dynamics. The two forms cannot be mixed within one mode block. In STLMC, a system of ODEs over continuous variables $x_1, \ldots, x_n$ is written as a sequence of equations of the following form, separated by semicolons. Each e_i is an expression over declared variables and constants:

```
d/dt[x_1] = e_1(x_1, ..., x_n) ;
...
d/dt[x_n] = e_n(x_1, ..., x_n) ;
```

For example, the following ODEs define the continuous dynamics $\dot{x} = \sin(y)$ and $\dot{y} = y^2$:

```
d/dt[x] = sin(y);  d/dt[y] = y ** 2;
```

A closed form flow is written as a set of continuous functions, parameterized by a time variable `t` and the initial values $x_1(0), \ldots, x_n(0)$ for the mode block:

```
x_1(t) = e_1(t, x_1(0),...,x_n(0)) ;
...
x_n(t) = e_n(t, x_1(0),...,x_n(0)) ;
```

Here `t` is elapsed time within the current continuous segment, and `x(0)` denotes that segment's initial value. A closed form flow must agree with its initial state at `t = 0` and remain

consistent when restarted at an STL variable point. The parser rejects detected constant initial-value contradictions and warns about some restart inconsistencies.

For example, the same variables may instead be defined by the following closed form expressions, $x(t) = 3t + x(0)$ and $y(t) = t + y(0)$:

```
x(t) = 3 * t + x(0);  y(t) = t + y(0);
```

A `jump` component may contain multiple rules of the form *guard* `=>` *reset*, each describing a possible transition. A transition is enabled when the current state satisfies its guard. The reset constrains primed variables, which represent the state after the jump, using expressions over current state variables and constants.

A reset should explicitly constrain every mode and continuous variable. To preserve a value, write `x' = x`; an omitted variable is unconstrained after the jump. For example, the following defines a jump with four variables `b`, `i`, `x`, and `y` (declared above):

```
(and (i = 1) (x > 10)) =>
  (and (b' = false) (i' = 3) (x' = x) (y' = 0));
```

4.3 Initial conditions

Every model must contain exactly one `init` section after its mode blocks. The section contains one or more Boolean state conditions separated by semicolons over declared mode and continuous variables. Arithmetic expressions in these conditions may also use numeric constants declared with `const`. The conditions are interpreted conjunctively and define the set of initial states.

Initial conditions use unprimed variables. A variable omitted from the `init` section is not assigned a default value; it remains unconstrained except for its declared domain and the conditions of the selected mode block. For example, the following constrains `b`, `i`, `x`, and `y`:

```
init:
not b;    i = 0;    19.9 <= x;    y = 0;
```

4.4 State propositions and STL properties

Named state propositions may be declared in the optional `proposition` section before the `goal` section. They provide short names for conditions that occur repeatedly in STL formulas. Each proposition defines a Boolean state condition; it cannot contain temporal operators or refer to another named proposition. For example:

```
proposition:
[p1]: x0 - x1 >= 4;
[p2]: x0 - x1 <= -3;
```

Every proposition referenced by an STL formula must be declared in the `proposition` section.

The required `goal` section declares the STL properties or reachability queries to be analyzed. An STL property may be declared with or without a label. A label is written in square brackets before the colon and uniquely identifies the property within the model file.[1] For example, the following declares two labeled STL properties from the running example using atomic conditions directly in the formulas:

```
goal:
[f1]: <>[0, 15](x0 >= 14 U[0, inf) x1 <= 19);
[f2]: [][2, 4]((x0 - x1 >= 4) -> <>[3, 10] (x0 - x1 <= -3));
```

The second formula can be written using the propositions declared above:

[1] Labels can be used to select individual goals for analysis; see Section 6.

```
goal:
[f2]: [][2, 4](p1 -> <>[3, 10] p2);
```

Boolean values and proposition aliases may also be compared with `=` and `!=`; for example, `[][0, 5](p1 = false)`. Boolean comparisons cannot mix numeric and Boolean operands.

The temporal operators are `[]` (always), `<>` (eventually), `U` (until), and `R` (release). Boolean conjunction and disjunction are written as `and` and `or`, negation as `~`, and implication as `->`; `true` and `false` are Boolean constants. Parentheses are recommended when Boolean and temporal operators are combined.

Temporal intervals may be open, closed, or half open. An unbounded interval uses `inf` as an open upper endpoint, as in `[0, inf)`. A singleton interval is written as `=c`, as in `[]=3 p1`.

In addition to STL properties, the `goal` section accepts an unlabeled goal written with the `reach` keyword followed by a state condition.[2]

```
goal:
reach (x >= 10);
```

4.5 Comments

A line comment begins with `#` and continues to the end of the line. A block comment begins and ends with three single quotation marks. Both forms may appear wherever whitespace is allowed in a model file.

```
# This is a line comment.
''' This is a
    block comment. '''
```

5 Analysis

This section provides an overview of the analyses supported by STLMC and shows how to run the tool and interpret its output. Two types of analysis are available:

- *Robust STL model checking* searches the bounded executions of a model for a violation of an STL property at the given robustness threshold.
- *Reachability analysis* asks whether a bounded execution can reach a state satisfying a given condition. A query can be declared in the model with `reach`; alternatively, `-reach` treats a selected state goal without temporal operators as a reachability target.

Optional analysis features, such as the two step procedure and parallel solving, can be enabled through command line arguments or configuration files. The SMT solver may also be specified explicitly; otherwise, STLMC selects a compatible solver automatically. Section 6 describes these settings, and Section 9 summarizes the solver capabilities.

5.1 Running an analysis

The command line must specify a model file. An analysis also requires a discrete bound and a time bound, which may be supplied through command line arguments or configuration files. Figure 4 shows the command line syntax; Section 6 gives the settings and precedence rules.

Before solving, STLMC prints the model, goal, solver, analysis method, execution mode, bounds, threshold, and query type. In model checking, the discrete bound limits the number of mode changes and STL variable points. In reachability analysis, it limits the number of jumps,

[2]The corresponding reachability analysis and the `reach` argument are described in Section 5.

```
$ stlmc [path to model file] \
        -bound [int] -time-bound [number] \
        -threshold [number] -goal [name] ...
```

Figure 4: Command line syntax for running an analysis.

so bound zero examines the initial continuous segment. The applicable bound kind is included in the printed settings.

Model checking finishes with `satisfied`, `violated`, or `unknown`. Reachability analysis instead reports `reachable`, `unreachable`, or `unknown`. A violating execution is a counterexample, while an execution reaching the target is a witness. An `unknown` result is inconclusive.

These outcomes concern the configured bounded search. With a positive threshold, `violated` can be reported even when the Boolean property holds: robustness 0.2, for example, fails a requested threshold of 1. Reachability targets are relaxed by the same threshold; with $\epsilon = 0.1$, a target $x \geq 5$ is checked as $x \geq 4.9$. Neither a satisfied result nor an unreachable result is an unbounded proof, and a model with no feasible executions may have no counterexample.

The global `time-bound` and per-segment `time-horizon` limit modeled time. Segment limits apply to all solvers and restrict the executions represented by the encoding.

One step and two step analysis can each use symbolic or explicit path exploration. Symbolic exploration leaves mode and jump choices in the SMT encoding, whereas explicit exploration enumerates transition sequences. Batching is available for explicit one step and two step analysis, whereas parallel execution applies only to two step refinement checks. These settings are described in Section 6.

When a counterexample or witness is found, `-visualize` writes result data and a visualization configuration. Section 7 describes these artifacts and their output formats. Generated solver queries can also be retained with `-save-smt2`; its output directory is selected with `-smt2-dir`.

For example, Figure 5 analyzes `f2` from the running example with maximum discrete bound 5, time bound 15, and threshold $\epsilon = 2$. The command uses dReal and the parallel two step method and finds a counterexample at bound 1. The time bound includes the full evaluation window of `f2`, which extends to time 14. Runtime and the selected execution can vary between runs.

```
$ stlmc ./thm.model -bound 5 -time-bound 15 \
        -threshold 2 -goal f2 -solver dreal \
        -two-step -parallel -visualize
STLmc - Signal Temporal Logic Model Checker
v1.0.2.dev1 (commit: 89b1264cc40b-dirty)
...
Result
  status      : violated at bound 1
  time        : 0.563s
  formula     : ([][2,4] (((x0 - x1) >= 4) ->
                  (<>[3,10] ((x0 - x1) <= (- 3)))))
  artifacts   : thm_b1_f2_dreal.counterexample
                thm_b1_f2_dreal.cfg
```

Figure 5: Abridged output for the analysis of `f2` in the running example.

6 Configuration

Analysis settings can be supplied as command line arguments or stored in configuration files. Command line arguments are convenient for individual runs, while configuration files provide reusable settings for a model or an individual goal. This section describes both forms and how their values are combined.

6.1 Command line arguments

The command line must specify a model file. The configuration must also provide `bound` and `time-bound`, either through command line arguments or through one of the configuration layers. All other configuration files and command line overrides are optional.

```
$ stlmc [path to model file] \
        -default-cfg [path to default config file] \
        -model-cfg [path to model config file] \
        -model-specific-cfg [path to goal config file] \
        -bound [int] ...
```

Most analysis settings have both a configuration parameter and a command line form. A command line argument is formed by adding a leading hyphen to the parameter name; for example, `bound` becomes `-bound`. Boolean arguments such as `-two-step`, `-parallel`, and `-save-smt2` enable the corresponding option. To disable a Boolean value inherited from a configuration file, set it to `"false"` in a configuration file of higher priority.

By default, STLmc analyzes every STL goal declared in the model. The `-goal` argument selects labeled goals; multiple labels can be separated by commas, as in `-goal f1,f2`. Other goals, including unlabeled goals and reachability queries, are skipped when labels are selected.

Individual command line values override every configuration file. Invalid solver and expression combinations are described in Section 9. The purpose and behavior of the analysis arguments are described in Section 5.

6.2 Configuration files

A configuration file has the extension `.cfg`. Parameters common to all solvers are declared in a `common` section, while parameters for individual solvers are declared in `z3`, `yices`, `cvc5`, and `dreal` sections. Figure 6 shows the default configuration included with STLmc, and Table 1 summarizes the `common` parameters.

6.2.1 Common parameters

The `common` section contains the parameters in Table 1: two required bounds and sixteen optional settings.

The parameter names are the command line argument names without their leading hyphen. Numeric values are written directly, while Boolean values are quoted as `"true"` or `"false"`. The `smt2-dir` value is used only when `save-smt2` is enabled.

The `path-strategy` parameter is independent of `two-step`, so all four combinations of one step or two step analysis and symbolic or explicit exploration are available. `solver-batch-size` limits the candidates combined in one solver query. The default `parallel-core = "auto"` uses all logical CPUs reported by the host; an explicit positive integer sets a worker limit. Scenario generation remains sequential, so actual concurrency can be lower than the worker limit. The optional `smt-preprocess` simplifies SMT definitions using assignments fixed by a scenario. `concrete` retains a complete Boolean and discrete assignment instead of applying unsat core generalization.

```
# basic section
common {
    # mandatory arguments
    # bound =
    # time-bound =

    threshold = 0.01
    solver = "auto"
    time-horizon = "time-bound"
    parallel = "false"
    visualize = "false"
    goal = "all"
    path-strategy = "symbolic"
    two-step = "false"
    concrete = "false"
    verbose = "false"               # print verbose messages
    reach = "false"
    parallel-core = "auto"
    solver-batch-size = 1
    save-smt2 = "false"
    smt-preprocess = "false"
    smt2-dir = "smt2-logs"
}

# underlying solver
z3      {
    logic = "QF_NRA" # "QF_LRA"
}

yices   {
    logic = "QF_NRA" # "QF_LRA"
}

cvc5 {
    logic = "QF_NRA" # "QF_LRA"
}
dreal {
    precision = 0.001
    ode-order = 20
    # ode-step is omitted to use dReal's automatic step control.
    executable-path = "dReal" # dReal3 executable name or path
}
```

Figure 6: The default configuration included with STLmc.

6.2.2 Parameters for individual solvers

The SMT solver is selected with the `solver` parameter. Solver capabilities and automatic selection are described in Section 9.

Parameters for an individual solver are written in a section named after it. The `z3`, `cvc5`, and `yices` sections each accept a `logic` parameter, either `QF_LRA` or `QF_NRA`. The `dreal` section accepts `precision`, `ode-order`, `ode-step`, and `executable-path`. The `ode-order` and `executable-path` parameters are mandatory in that section. `precision` sets the precision δ for δ-complete solving. `ode-order` sets the maximum Taylor expansion order, and `executable-path` gives either the dReal3 executable name or its path. With the default value `dReal`, STLmc searches its managed user solver directory and then `PATH`. An explicit executable path takes precedence. The optional `ode-step` sets the numerical integration step; when it is omitted, dReal uses automatic step control.

Table 1: Parameters in the `common` section.

Name	Explanation	Default
`bound`	a discrete bound $N \in \mathbb{N}$	-
`time-bound`	a time bound $\tau \in \mathbb{Q}^+$	-
`threshold`	a robustness threshold $\epsilon \in \mathbb{Q}^+$	0.01
`solver`	`auto`, `z3`, `cvc5`, `yices`, or `dreal`	`auto`
`time-horizon`	a continuous segment duration bound	τ
`goal`	a list of STL goals to be analyzed	all
`path-strategy`	symbolic or explicit path exploration	`symbolic`
`two-step`	use the two step optimization	disabled
`parallel`	parallelize two step analysis	disabled
`parallel-core`	maximum parallel refinement workers	`auto`
`visualize`	generate counterexample or witness data	disabled
`concrete`	retain complete scenarios in two step analysis	disabled
`reach`	check reachability instead of the negated STL goal	disabled
`solver-batch-size`	candidates combined in one solver query	1
`smt-preprocess`	simplify scenario-fixed SMT definitions	disabled
`save-smt2`	save generated SMT-LIB2 queries	disabled
`smt2-dir`	output directory used by `save-smt2`	`smt2-logs`
`verbose`	print more information about the execution	disabled

6.2.3 Configuration precedence

Parameters are combined in the following order, with each later source overriding values from earlier sources:

1. the default configuration included with STLmc;
2. a model configuration;
3. a model specific configuration; and
4. command line options.

The `default.cfg` file installed with the STLmc package provides every default except the mandatory `bound` and `time-bound`. A file supplied with `-default-cfg` replaces this default file. It must therefore provide all required sections and all mandatory solver parameters, in addition to values that may be supplied by later configuration layers.

Without `-model-cfg`, STLmc looks beside the model file for the configuration name formed from the part of the model file name before its first period; for example, `thm.model` causes `thm.cfg` to be loaded when that file exists. A model specific file is loaded only when it is supplied with `-model-specific-cfg`; it is not discovered automatically.

The model configuration stores reusable settings for a particular model. In the running example, `thm.cfg` sets bound 5, time bound 15, dReal, and the parallel two step algorithm for the model in Figure 2. Figure 7 shows this partial configuration; all omitted parameters retain their values from the default configuration.

```
# STLmc configuration
common { bound = 5
         time-bound = 15
         solver = "dreal"
         verbose = "true"
         two-step = "true"
         parallel = "true" }
```

Figure 7: A model configuration example.

```
# STLmc configuration
common { goal = f2
         threshold = 2
         time-horizon = 10 }
```

Figure 8: A model specific configuration example.

A model specific configuration can override selected analysis parameters for an individual goal. For example, the configuration in Figure 8 selects the goal `f2`, sets $\epsilon = 2$, and limits each continuous segment to 10 time units within the global time bound of 15. All other values are inherited from the model configuration and the default configuration.

With this segment limit, the configuration example reports a counterexample at bound 2. It is a different bounded query from Figure 5, whose default segment limit equals the global time bound of 15.

7 Counterexample and witness analysis

With `-visualize` enabled, STLmc records a violating model checking execution in a result file with the `.counterexample` extension. A successful reachability execution uses the `.witness` extension. The `stlmc-vis` command can inspect either artifact in two ways: it can export a textual account of the execution or visualize the state and robustness trajectories.

Result files contain versioned, gzip-compressed JSON. Older pickle artifacts are unsupported; rerun the analysis with the current tool to regenerate them. Rendering saved results does not require solver bindings.

7.1 Text output

Text output provides a readable account of the counterexample without a visualization configuration file:

```
$ stlmc-vis [path to result file] -output text
```

The command appends `.txt` to the result file name. For example, it writes `thm_b1_f2_drea l.counterexample.txt`:

```
$ stlmc-vis thm_b1_f2_dreal.counterexample -output text
```

Figure 9 shows an excerpt from a generated file. Its header identifies the goal, expanded formula, bound, and robustness threshold (printed as `delta`; see Section 9). Subsequent sections list the declared constants, domains of continuous variables, and initial condition. Each `Bound` block records one trajectory segment: its time interval, active mode, invariant, continuous dynamics, and state values at the two endpoints. Except for the last segment, the block also lists the next mode and the jump candidates from the current segment to the next.

For example, the first segment in Figure 9 runs from time 0 to 13 with both heaters off. Over this segment, x_0 changes from approximately 20.72 to 17.14, while x_1 changes from approximately 18.72 to 10.86. The arrow separates the value at the start of the segment from its value at the end. Both heaters remain off in the next segment, which ends at time 15: the boundary is an STL partition point, not a heater switch. Each line containing `=>` pairs an available jump guard with its reset; these candidates are not a record of a jump actually taken. Ellipses in the figure mark content omitted from the displayed excerpt; they are not part of the generated file.

```
Counterexample
==============
goal: f2
formula: ([][2,4] (((x0 - x1) >= 4) ->
  (<>[3,10] ((x0 - x1) <= (- 3)))))
bound: 1
delta: 2.0
Constants
---------
c0 = 0.98
c1 = 0.97
...
Variables
---------
x0: [10.0, 35.0]
x1: [10.0, 35.0]
Initial condition
-----------------
(and (on0 = 0) (18 <= x0) (x0 <= 22)
  (on1 = 0) (18 <= x1) (x1 <= 22))
Bound 0
-------
time: 0.0 -> 13.0
mode: (and (on0 = 0) (on1 = 0))
invariant:
  (and (x0 > 10) (x1 > 10))
flow:
  d/dt[x0] = ((- 0.015) * ((0.98 * x0) - (0.01 * x1)))
  d/dt[x1] = ((- 0.045) * ((0.97 * x1) - (0.03 * x0)))
state:
  x0: 20.721074425072878 -> 17.14202417522523
  x1: 18.721074425072878 -> 10.86483036977684
next mode: (and (on0 = 0) (on1 = 0))
jump candidates:
  (x0 <= 17) => (and (on0' = 1) (on1' = on1)
    (x0' = x0) (x1' = x1))
  (x1 <= 16) => (and (on1' = 1) (on0' = on0)
    (x0' = x0) (x1' = x1))
...
```

Figure 9: An excerpt from a text counterexample file.

7.2 Visualization

HTML and PDF output plot the state and robustness trajectories in a counterexample. Both formats use a visualization configuration file:

```
$ stlmc-vis [path to result file] \
            -cfg [path to configuration file] \
            -output [html|pdf]
```

If `-cfg` is omitted, `stlmc-vis` automatically looks in the current directory for a configuration file whose name matches the result file. For example, `thm_b1_f2_dreal.counterexample` uses `thm_b1_f2_dreal.cfg`. The `-output` argument overrides the format recorded in that file. Generated configurations default to HTML. HTML output uses Bokeh, and PDF output uses Matplotlib; install the desired renderer as described in Section 2.

For dReal results, plots reconstruct the execution from interval midpoints. These coordinates need not form an exact trajectory satisfying every flow, invariant, and reset. Sampled robustness and plots aid inspection but do not provide an independent continuous-time certificate. Interactive HTML plots load Bokeh JavaScript from a CDN and may require internet access.

The visualization configuration records:

- the continuous variables and STL subformula labels available for plotting;
- the output format; and

- groups of variables to be plotted together.

Each group is plotted on one graph and must contain either state variables or STL subformula robustness curves. Both have numerical values, but the two categories cannot be mixed: for example, `(x0, f2_0)` is rejected. Items not assigned to a group are collected separately into a state group and a robustness group. Figure 10 shows the thermostat visualization configuration.

Consider the counterexample for `f2` generated by the analysis in Figure 5 and the visualization configuration below. The following command generates the PDF plots in Figure 11. The example directory includes a `matplotlibrc` file that enlarges plot labels for printing.

```
$ stlmc-vis thm_b1_f2_dreal.counterexample -cfg thm_vis.cfg -output pdf
```

```
{   # continuous variables: x0, x1
    # STL subformula labels:
    # f2_0 --> [][2, 4]((x0 - x1 >= 4) -> <>[3,10](x0 - x1 <= -3))
    # f2_1 --> (x0 - x1 >= 4) -> <>[3,10](x0 - x1 <= -3)
    # f2_2 --> not (x0 - x1 >= 4)
    # f2_3 --> <>[3,10](x0 - x1 <= -3)
    # f2_4 --> x0 - x1 <= -3
    # f2_5 --> x0 - x1 >= 4

    output = pdf # html
    group { (x0, x1), (f2_0, f2_1), (f2_2, f2_3), (f2_4, f2_5) }
}
```

Figure 10: A visualization configuration example.

At time 0, the sampled robustness of `f2` is approximately 0.28, below the requested threshold $\epsilon = 2$. Thus this execution fails the robustness requirement even though its sampled Boolean value at time 0 is true. Between time 2 and time 4, the implication `f2_1` falls below the threshold. Its eventual condition `f2_3` examines times from 5 to 14. The plot legends use the subformula labels from Figure 10; `f2_4` and `f2_5` denote the two atomic conditions.

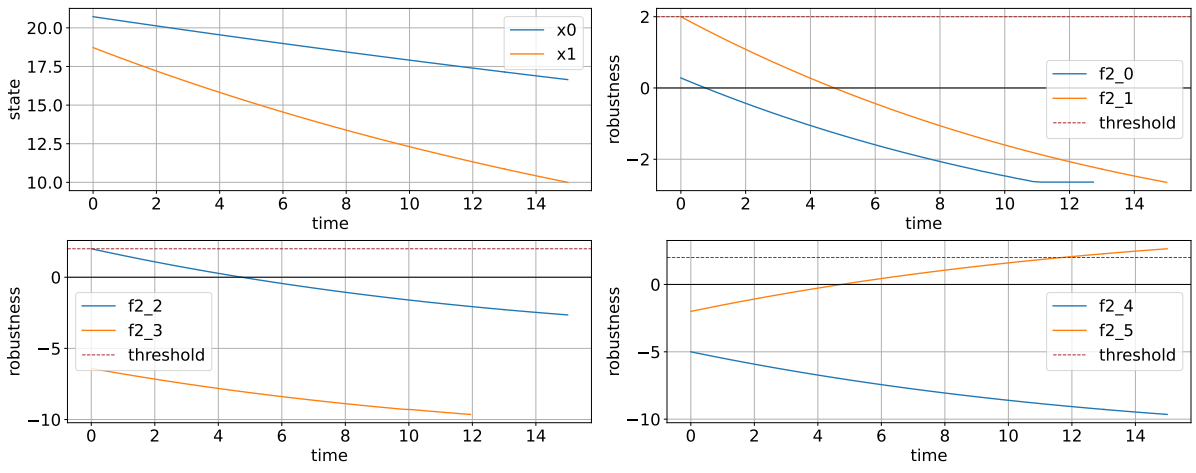


Figure 11: Plots generated from the counterexample in Figure 5. Dashed horizontal lines mark $\epsilon = 2$.

8 Browser workspace

The browser workspace provides editors for models and configurations, analysis controls, and result inspection. It uses the same parser and configuration defaults as the command line tool and supports Linux, macOS, and WSL. Figure 12 shows the model and configuration editors, analysis settings, and results for the GUI quick-start example.

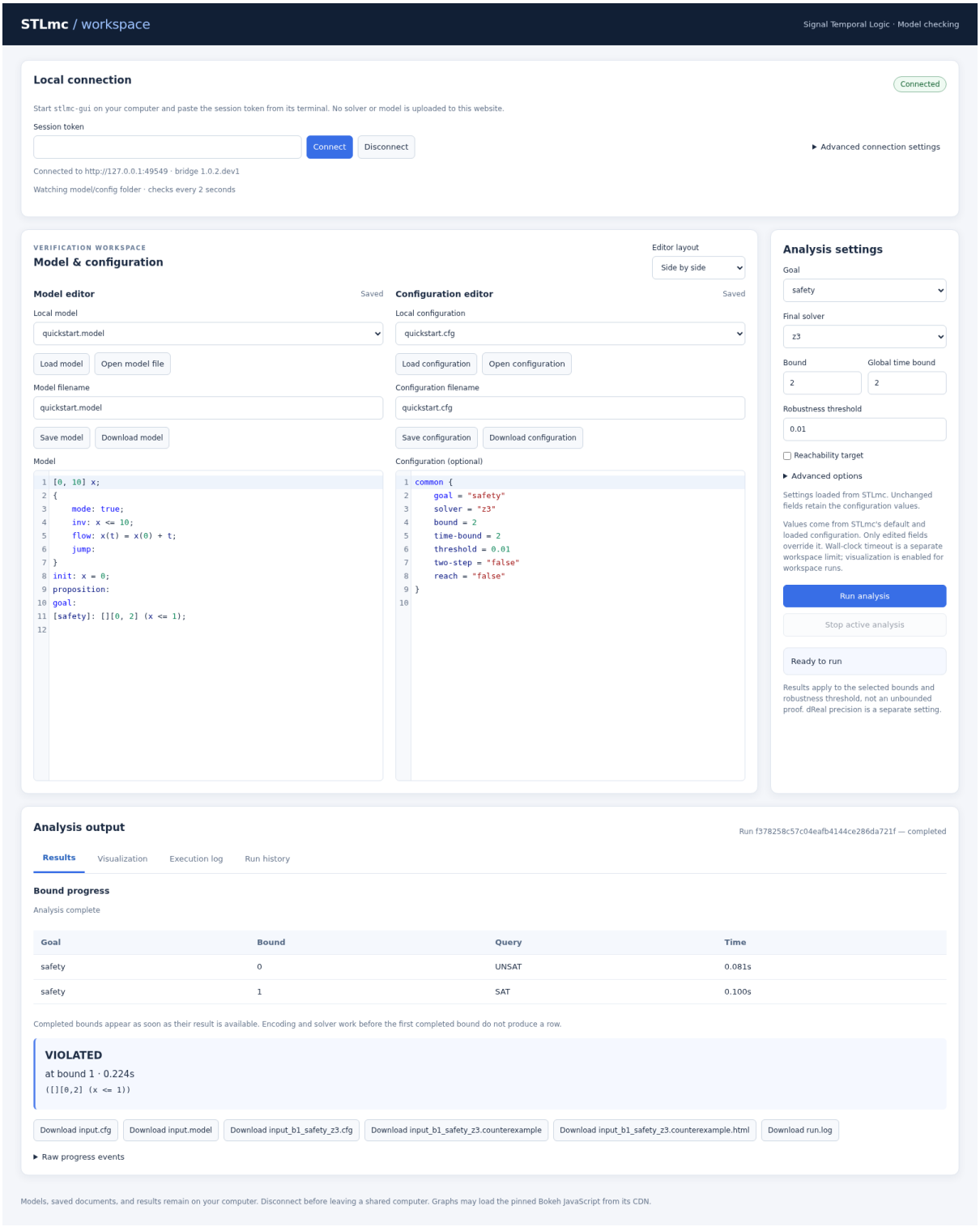


Figure 12: The browser workspace displaying the quick-start model and its verification result.

8.1 Starting the workspace

Install the optional GUI dependencies together with a solver and HTML visualization, then start the workspace:

```
$ stlmc-installer install --solver z3 --visualize html --ui
$ stlmc-gui
```

The browser opens at `http://127.0.0.1:8766/`. Enter the session token printed in the terminal and click *Connect*. Keep that terminal open. The bridge API uses port 8765; it is not the workspace page. The `--ui-port` and `--port` options change the page and API ports. For a trusted local session, `--no-auth` enables automatic connection; keep authentication enabled on shared machines.

8.2 Editing and running analyses

Select a local model and click *Load model*. Load a configuration separately with *Load configuration*. File upload reads a file into the editor; *Save* writes the buffer to the server's model folder, and *Download* saves a copy on the browser's computer. Parser diagnostics appear beside the editors and link to the relevant text.

Configuration edits apply automatically. Only fields explicitly changed in *Analysis settings* override the configuration; loading a configuration resets these overrides. Form edits do not rewrite the configuration text. Supply the discrete and global time bounds in the form or configuration. Settings without form controls, including solver precision, parallelism, and `time-horizon`, remain controlled by the configuration.

Click *Run analysis* to use the current buffers; saving first is optional. The GUI enables result generation and applies a separate computation timeout, which defaults to 120 seconds and can be set up to 3600 seconds. This timeout differs from modeled time bounds. Later edits do not change an active run. Use *Stop active analysis* to cancel it; closing the page does not cancel an active analysis.

8.3 Inspecting and keeping results

Results lists outcomes, timings, and downloadable artifacts. *Visualization* displays counterexamples or witnesses, and *Execution log* shows solver output and diagnostics. Result meanings are the same as in Section 5; a completed process does not imply a satisfied property, and a timeout is inconclusive.

Run history reopens earlier results without replacing the editor buffers. Clearing finished history permanently deletes the listed finished runs and their artifacts. Download needed results first. Models, configurations, and active runs are retained.

Models and run history reside in user data folders outside the package. Use `--models-dir` and `--runs-dir` to choose separate folders. The workspace detects external file changes; clean buffers reload, while unsaved edits are preserved with a conflict notice. A GUI instance is a shared workspace intended for one user. Remote access and further options are described in the GUI guide.

9 SMT solver support

STLmc supports Z3 [2], CVC5, Yices2 [3], and dReal [4]. Compatibility with a solver depends on the expressions that appear throughout the model file, including expressions in flows, invariants, jump conditions, initial conditions, state propositions, and STL properties. Table 2 summarizes the expression constructs that STLmc can translate for each solver.

Table 2: Model expression constructs supported by each SMT solver.

Category	Operators and functions	Z3	CVC5	Yices2	dReal3
Boolean operators	`and`, `or`, `not`, `~`	✓	✓	✓	✓
Comparisons	`>`, `>=`, `<`, `<=`, `=`, `!=`	✓	✓	✓	✓
Arithmetic	`+`, `-`, `*`, `/`, and unary `-`	✓	✓	✓	✓
Constant powers	`x ** n`, for a constant integer `n >= 0`	✓	✓	✓	✓
Other powers	`x ** y`, for a fractional, negative, or symbolic `y`	–	–	–	✓
Functions	`sqrt`, `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`	–	–	–	✓

In Table 2, a check mark indicates that STLMC can translate the construct for the solver; a dash indicates that STLMC does not support the combination. A check mark does not guarantee that every resulting nonlinear problem can be solved within a particular time. Division by an expression that may be zero is also subject to the arithmetic semantics of the selected solver. Z3, CVC5, and Yices2 accept only constant integer powers greater than or equal to zero; fractional, negative, and symbolic powers require dReal.

9.1 Solver selection and compatibility

With `solver = "auto"`, STLMC examines the model and selected goals. For algebraic queries it tries Z3, CVC5, Yices2, and then dReal, selecting the first supported backend that passes a runtime check. A general ODE depending on evolving state or time, a transcendental function, or a fractional, negative, or symbolic exponent restricts automatic selection to dReal. Autonomous state-independent ODEs can be considered by algebraic backends. The configured arithmetic logic is checked for compatibility; a nonlinear query cannot use `QF_LRA`. Automatic selection does not install solvers.

When a solver is selected explicitly, every expression in the model and the selected goal must be supported by that solver. If they are incompatible, the analysis cannot proceed; users should manually select a compatible solver from Table 2 or use `solver = "auto"`.

Z3, CVC5, and Yices2 can translate supported autonomous polynomial ODEs with acyclic state dependencies to exact closed form constraints. Other ODEs require dReal's native ODE support. Explicitly selecting an algebraic backend can therefore support some ODEs for which automatic selection chooses dReal.

For a disjunctive invariant, different branches may hold at different times within one segment. Support depends on both the flow and backend: Z3 and CVC5 can retain a universal time condition for supported flows, while Yices2 and dReal require time elimination. Unsupported combinations produce an error rather than a solver `unknown` result.

Internal scenario and unsat core reasoning use Z3 by default, independently of the final solver. Advanced users can select Z3, CVC5, or Yices2 through `STLMC_SCENARIO_SOLVER` and `STLMC_CORE_SOLVER` environment variables. Required backends must be installed separately.

dReal's `precision` is a delta-decision tolerance, not the STL robustness threshold. A delta-SAT answer need not satisfy the exact unrelaxed constraints. The official dReal3 release also has known invariant and ODE witness limitations; neither installation probes nor sampled visualization certify general ODE soundness. See the result semantics guide for interpretation and validation limits.

9.2 dReal function names

The model language uses `arcsin`, `arccos`, and `arctan`. STLMC translates them to `asin`, `acos`, and `atan`, respectively, for the SMT-LIB2 syntax accepted by dReal3. For compatibility with that solver, `tan(x)` is translated to `sin(x) / cos(x)`.

References

[1] Kyungmin Bae and Sicun Gao. Modular smt-based analysis of nonlinear hybrid systems. In *Proceedings of the 17th Conference on Formal Methods in Computer-Aided Design*, FMCAD '17, pages 180–187, Austin, TX, 2017. FMCAD.

[2] Leonardo De Moura and Nikolaj Bjørner. Z3: an efficient SMT solver. In *Proc. TACAS*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008.

[3] Bruno Dutertre. Yices 2.2. In Armin Biere and Roderick Bloem, editors, *Proc. CAV*, volume 8559 of *LNCS*, pages 737–744. Springer, 2014.

[4] Sicun Gao, Soonho Kong, and Edmund M. Clarke. dReal: An SMT solver for nonlinear theories over the reals. In *Proc. CADE*, volume 7898 of *LNCS*, pages 208–214. Springer, 2013.

[5] Thomas A. Henzinger. The theory of hybrid automata. In *Verification of Digital and Hybrid Systems*, volume 170 of *NATO ASI Series*, pages 265–292. Springer, 2000.

[6] Soonho Kong, Sicun Gao, Wei Chen, and Edmund M. Clarke. dReach: δ-reachability analysis for hybrid systems. In *Proc. TACAS*, volume 7898 of *LNCS*, pages 200–205. Springer, 2015.

[7] Geunyeol Yu, Jia Lee, and Kyungmin Bae. STLmc: Robust STL model checking of hybrid systems using SMT. In *Computer Aided Verification*, volume 13371 of *Lecture Notes in Computer Science*, pages 524–537. Springer, 2022.